

UTS

Sistem Paralel dan Terdistribusi A

**Pub-Sub Log Aggregator dengan Idempotent Consumer
dan Deduplication**



Disusun Oleh :

Mahardika Arka 11231037

23 April 2026

Tautan Terkait

Link Video Youtube Tutorial : <https://www.youtube.com/watch?v=IUz1zi9Rfyo>

Link Repository Github: https://github.com/spirinity/sister_uts

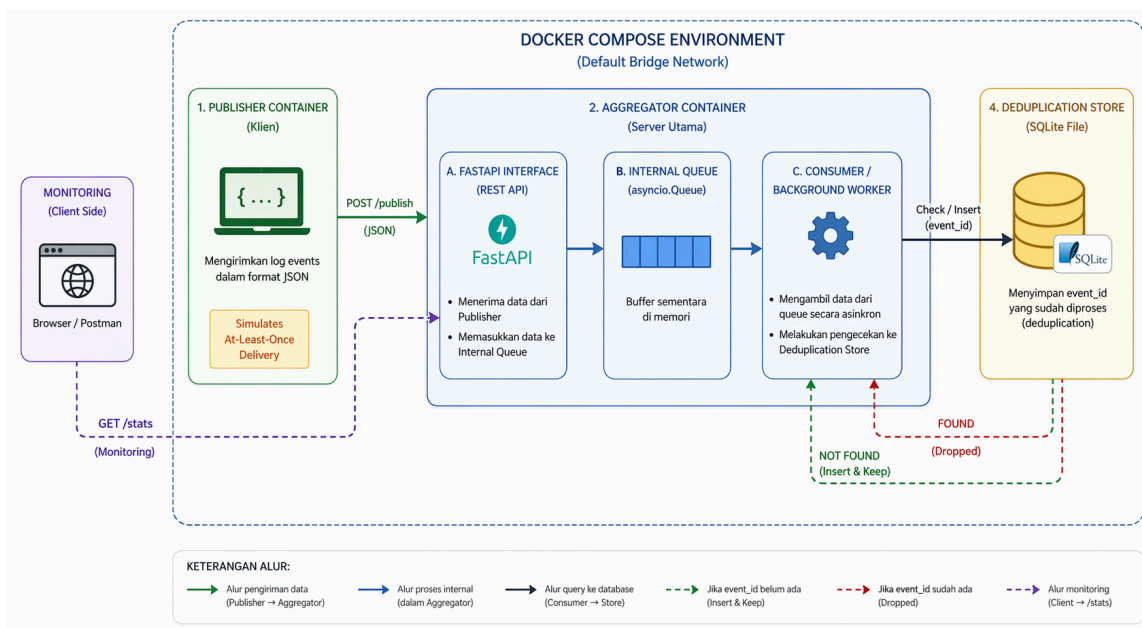
Ringkasan Sistem dan Arsitektur

Layanan *Pub-Sub Log Aggregator* ini adalah sebuah prototipe sistem terdistribusi yang dirancang untuk mengumpulkan, menyaring, dan menyimpan data log (*events*) secara efisien. Sistem ini mengadopsi pola arsitektur *Publish-Subscribe* untuk memastikan pemisahan (*decoupling*) yang optimal antara komponen pengirim pesan dan komponen pengolah data.

Secara arsitektur, sistem ini dibagi menjadi dua entitas layanan yang terisolasi yaitu *Publisher* dan *Aggregator*. *Publisher* bertindak sebagai klien independen yang mensimulasikan pengiriman *event* dengan skenario *at-least-once delivery*, di mana pesan yang sama sengaja dikirimkan berulang kali untuk menguji keandalan sistem. Di sisi *Aggregator*, data diterima melalui antarmuka REST API berbasis FastAPI dan langsung dimasukkan ke dalam *in-memory queue* (*asyncio.Queue*) sebagai broker internal.

Sebuah *background worker* (*Consumer*) bertugas menarik data dari antrian tersebut dan memprosesnya menggunakan prinsip *Idempotent Consumer*. Artinya, sistem menjamin hasil akhir pemrosesan tetap konsisten meski menerima pesan ganda. Sebelum disimpan secara permanen, setiap *event* divalidasi ke dalam *Deduplication Store* berbasis basis data relasional lokal (*SQLite*) menggunakan *composite key* dari topic dan *event_id* untuk menolak duplikasi secara mutlak dan memastikan data tetap persisten meskipun terjadi *crash* atau *restart*.

Untuk menjamin kemudahan reproduksi (*reproducibility*) dan isolasi jaringan, seluruh ekosistem ini dikontainerisasi menggunakan Docker. Layanan *Publisher* dan *Aggregator* diorkestrasikan bersama menggunakan Docker Compose di dalam satu jaringan internal default, memastikan komunikasi sistem berjalan sepenuhnya secara lokal tanpa ketergantungan pada layanan eksternal publik.



Gambar Diagram Arsitektur Sistem

Tabel List Endpoint API

Method	Endpoint	Deskripsi
POST	/publish	Kirim satu atau banyak (batch) event
GET	/events	Lihat daftar event unik yang sudah diproses
GET	/stats	Lihat statistik sistem secara real-time

Keputusan Desain

Dalam membangun layanan *Pub-Sub Log Aggregator* ini, beberapa keputusan desain arsitektural diambil untuk memenuhi batasan lingkungan lokal dan menjamin keandalan sistem terhadap kegagalan.

A. Idempotency dan Deduplication Store

- **Keputusan:** Menggunakan *database* relasional lokal (SQLite) sebagai *deduplication store*, alih-alih menggunakan struktur data di dalam memori (*in-memory* seperti *Dictionary* atau *Set*).
- **Alasan Teknis:** Mekanisme deduplikasi diimplementasikan dengan memanfaatkan fitur *Composite Primary Key* (atau *Unique Constraint*) pada kombinasi kolom *topic* dan *event_id* di tabel SQLite. Ketika *Consumer* mencoba menyimpan *event* duplikat, *database* akan menghasilkan *Integrity Error*. Aplikasi dikonfigurasi untuk menangkap (*catch*) *error* tersebut, mencatatnya di log sistem sebagai *duplicate dropped*, dan melanjutkan operasi tanpa menghentikan sistem. Pendekatan ini menjamin sifat *Idempotent Consumer*, di mana pemrosesan *event* yang sama berkali-kali tidak akan mengubah status (*state*) akhir sistem.

B. Ordering (Pengurutan)

- **Keputusan:** Sistem **tidak** menerapkan *total ordering* (pengurutan mutlak) secara global di tingkat antrean, melainkan menggunakan pendekatan berbasis *timestamp* bawaan dari *payload* (ISO8601).
- **Alasan Teknis:** Dalam konteks agregator log, *event* sering kali datang dari berbagai sumber (*source*) yang berbeda secara asinkron. Memaksakan *total ordering* pada saat *event* diterima akan menciptakan *bottleneck* performa yang signifikan dan menurunkan *throughput*. Oleh karena itu, agregator menerima *event* apa adanya (*out-of-order* diperbolehkan), dan mengandalkan *timestamp* internal dari *event* tersebut. Jika pengurutan ketat dibutuhkan untuk analisis, hal tersebut dapat dilakukan pada tahap pembacaan di *endpoint* GET /events.

C. Reliability dan Toleransi Crash (At-Least-Once Delivery)

- **Keputusan:** Pemisahan proses (*decoupling*) menggunakan *asyncio.Queue* dan penyimpanan *state* ke dalam fail fisik pada *disk* (SQLite).
- **Alasan Teknis:** Untuk mengantisipasi skenario pengiriman *at-least-once delivery* (di mana *Publisher* mungkin melakukan pengiriman ulang karena tidak menerima *acknowledgement*), antrean internal memungkinkan API merespons dengan cepat



sehingga menekan angka *timeout* di sisi *Publisher*. Selain itu, karena *state* pemrosesan dicatat di dalam fail basis data lokal (bukan sekadar di RAM), sistem memiliki toleransi kegagalan (*crash tolerance*). Apabila *container* Docker mengalami interupsi atau di-*restart*, riwayat *event_id* yang telah diproses tetap persisten, sehingga *Consumer* tidak akan memproses ulang *event* lama yang dikirimkan kembali pasca-*restart*.

Analisis Performa dan Metrik

A. Skenario Pengujian (Stress Test)

Pengujian performa dilakukan menggunakan skrip *automated test* (test_stress.py) yang menembakkan *request* secara simultan ke *endpoint* POST /publish. Parameter beban uji ditetapkan untuk menguji ketahanan dan sistem deduplikasi:

- Total Event Dikirim: 5.000 *events* secara total.
- Tingkat Duplikasi: Ditetapkan sebesar 20%, di mana 1.000 *events* merupakan duplikat dari kumpulan 4.000 *events* unik.
- Metode: Pengiriman data dari klien dilakukan dalam bentuk *batch* yang berisi 100 *event* per *request* untuk menguji efisiensi penerimaan sistem. Batas waktu maksimum (*timeout*) pemrosesan wajar ditetapkan pada 120 detik.

B. Hasil Metrik (Observability)

Berdasarkan pemantauan metrik *real-time* dari *endpoint* statistik, diperoleh validasi data sebagai berikut:

- **received (Total Diterima):** 5.000.
- **unique_processed (Lolos Dedup & Disimpan):** 4.000.
- **duplicate_dropped (Ditolak karena Duplikat):** 1.000.

C. Analisis Sistem

1. Throughput dan Responsivitas API: Sistem berhasil menerima seluruh 5.000 data dan menyelesaikan proses agregasi tanpa melewati batas waktu 120 detik. FastAPI yang dikonfigurasi dapat segera mengembalikan status 202 Accepted per *batch* membuktikan bahwa mekanisme *queue* internal terbukti membebaskan *publisher* dari *bottleneck* proses *database*.
2. Akurasi Deduplikasi (Idempotency): Sistem memenuhi prinsip validasi yang diwajibkan di mana jumlah *event* unik diproses ditambah *event* duplikat yang ditolak sama dengan total *event* diterima ($unique_processed + duplicate_dropped == received$). Logika *consumer* dapat membuang tepat 1.000 *event* duplikat (20%) tanpa terhenti (*crash*).
3. Ketahanan Konkurensi: Keandalan pencatatan statistik sistem selama menahan *burst traffic* ini sangat stabil karena pembaruan nilai variabel menggunakan mekanisme *thread-safe* dengan pelindung *asyncio.Lock*. Hal ini memitigasi kemungkinan adanya *race condition* antara proses penulisan *event* baru dengan penambahan nilai metrik *dropped* atau *processed*.

Keterkaitan dengan Buku

1. T1 (Bab 1): Karakteristik Sistem Terdistribusi dan Trade-off pada Pub-Sub

Aggregator

Sistem terdistribusi secara mendasar didefinisikan sebagai sekumpulan komputer independen yang tampil bagi penggunaanya sebagai satu sistem tunggal yang koheren (Tanenbaum & Van Steen, 2007). Karakteristik utamanya meliputi komponen yang beroperasi secara otonom (*autonomous nodes*), ketiadaan *global clock* untuk mengurutkan peristiwa secara absolut, dan kerentanan terhadap kegagalan komponen secara parsial atau *independent failures* (Tanenbaum & Van Steen, 2007).

Dalam perancangan layanan *Pub-Sub log aggregator* ini, karakteristik tersebut memunculkan *trade-off* desain yang signifikan. Penggunaan pola *publish-subscribe* menawarkan keunggulan berupa pemisahan waktu dan ruang atau *space and time decoupling* (Tanenbaum & Van Steen, 2007). Pada implementasinya, *Publisher* cukup mengirim *event* ke antarmuka FastAPI (`src/main.py`), lalu data langsung ditampung di memori internal (`asyncio.Queue` pada `src/queue_manager.py`). *Publisher* tidak perlu menunggu *Consumer* selesai memproses, sehingga sistem sangat responsif.

Namun, *trade-off* dari pemisahan ini adalah meningkatnya kompleksitas untuk menangani *independent failures* pada jaringan. Kegagalan komunikasi sering memicu pengiriman ulang (*retries*) yang menghasilkan *event* duplikat. Untuk mengatasinya, sistem mengorbankan sedikit kecepatan dengan menambahkan *Deduplication Store* berbasis SQLite (`src/dedup_store.py`). *Consumer* (`src/consumer.py`) diwajibkan mengecek *composite key* (`topic, event_id`) sebelum memproses pesan, menjadikannya sebuah *idempotent consumer*. Keputusan ini secara efektif menukar potensi duplikasi akibat kegagalan jaringan dengan jaminan konsistensi data.

2. T2 (Bab 2) Arsitektur Client-Server vs Publish-Subscribe untuk Aggregator

Arsitektur sistem terdistribusi umumnya dikategorikan berdasarkan cara komponen berinteraksi dan terorganisasi. Arsitektur *client-server* tradisional bersifat tersentralisasi di mana *client* mengirimkan permintaan layanan secara langsung kepada *server* yang menunggu (Tanenbaum & Van Steen, 2007). Sebaliknya, arsitektur *publish-subscribe* (Pub-Sub) menawarkan tingkat fleksibilitas yang lebih tinggi melalui *space and time decoupling* (Tanenbaum & Van Steen, 2007). Dalam Pub-Sub, pengirim (*publisher*) tidak perlu mengetahui identitas penerima (*subscriber*), dan keduanya tidak harus aktif secara bersamaan untuk bertukar pesan.

Pada proyek ini, pemilihan arsitektur Pub-Sub untuk *log aggregator* didasarkan pada kebutuhan skalabilitas dan ketahanan terhadap beban mendadak (*burst traffic*). Jika menggunakan *client-server* murni, setiap *log* yang masuk akan langsung diproses ke basis data secara sinkron, yang dapat menyebabkan *bottleneck* saat volume data tinggi. Dengan Pub-Sub yang diimplementasikan melalui `asyncio.Queue` di `src/queue_manager.py`, sistem

berhasil memisahkan proses penerimaan data di *endpoint* POST /publish (src/main.py) dengan proses pengolahan di src/[consumer.py](#).

Arsitektur Pub-Sub dipilih karena memungkinkan *aggregator* tetap responsif terhadap *publisher* meskipun proses penyimpanan ke SQLite membutuhkan waktu. Secara teknis, pemisahan layanan ini juga terlihat jelas dalam docker-compose.yml, di mana kontainer publisher dan aggregator beroperasi secara otonom dalam jaringan internal pubsub-net. Pola ini memastikan bahwa kegagalan sementara pada proses pengolahan data tidak akan menghentikan kemampuan sistem dalam menerima *event* baru, sehingga mencapai efisiensi yang jauh lebih baik dibandingkan model *client-server* sinkron (Tanenbaum & Van Steen, 2007).

3. T3

Dalam komunikasi sistem terdistribusi, *delivery semantics* mendefinisikan tingkat jaminan pengiriman pesan antar-komponen. Semantik *at-least-once delivery* menjamin bahwa sebuah pesan pasti akan dikirimkan ke penerima, namun mekanisme pengulangan otomatis (*retries*) akibat kegagalan jaringan atau *timeout* dapat menyebabkan pesan yang sama diterima berkali-kali (Tanenbaum & Van Steen, 2007). Sebaliknya, *exactly-once delivery* adalah jaminan ideal di mana pesan dipastikan sampai dan dieksekusi tepat satu kali, meskipun dalam praktiknya jaminan ini sangat sulit dicapai secara murni di tingkat jaringan karena membutuhkan protokol yang rumit (Tanenbaum & Van Steen, 2007).

Untuk menyeimbangkan antara keandalan dan performa, *log aggregator* pada proyek ini menerima semantik *at-least-once* di sisi jaringan (di mana *Publisher* mensimulasikan pengiriman duplikat), tetapi mendelegasikan tanggung jawab penyaringan ke sisi *backend* melalui penerapan *idempotent consumer*. Pada implementasi src/consumer.py, sifat *idempotent* ini menjadi sangat krusial dalam menghadapi *retries*. Tanpa sifat ini, pengiriman ulang sebuah *event* akan menghasilkan pencatatan ganda dan merusak integritas data metrik.

Sistem secara eksplisit memanggil fungsi `is_duplicate()` ke *database* SQLite (src/dedup_store.py) setiap kali *background worker* menarik pesan dari antrean. Jika `event_id` sudah pernah diproses, *Consumer* akan membuang pesan tersebut (*dropped*) dan hanya memprosesnya jika belum ada (Tanenbaum & Van Steen, 2007). Kombinasi *at-least-once delivery* dengan *idempotent consumer* ini secara efektif menghasilkan *exactly-once processing*, mencapai konsistensi data akhir yang sempurna tanpa harus membebani lapisan komunikasi dengan tingkat kompleksitas yang berlebihan.

4. T4 (Bab 4): Rancang Skema Penamaan dan Dampaknya terhadap Deduplikasi

Dalam sistem terdistribusi, penamaan (*naming*) berfungsi esensial untuk mengidentifikasi entitas agar dapat diakses, dikelola, dan dibedakan dari entitas lainnya (Tanenbaum & Van Steen, 2007). Salah satu konsep terpenting dalam bab ini adalah penggunaan *identifier* (pengenal pengenal sejati), yaitu sebuah nama berupa *flat naming*

(seperti bit string acak) yang mengacu tepat pada satu entitas dan sangat sulit mengalami tabrakan atau *collision* (Tanenbaum & Van Steen, 2007).

Pada layanan agregator log yang dibangun, skema penamaan direpresentasikan melalui definisi model pada `src/models.py`. Sistem menggunakan kombinasi dua tingkat: *topic* sebagai penamaan logis/terstruktur untuk mengelompokkan aliran log, dan *event_id* (direkomendasikan menggunakan format UUID) sebagai *flat identifier* yang *collision-resistant*. Kombinasi keduanya menghasilkan sebuah identitas komposit yang dijamin unik secara global melintasi berbagai layanan asal (*source*).

Dampak skema penamaan ini terhadap mekanisme deduplikasi sangatlah fundamental. Seperti yang terlihat pada implementasi `src/dedup_store.py`, sistem mengandalkan kombinasi (*topic*, *event_id*) sebagai *Composite Primary Key* pada tabel SQLite. Keandalan fungsionalitas deduplikasi sepenuhnya bersandar pada asumsi bahwa tidak akan ada dua *event* berbeda yang memiliki pengenalan yang sama. Jika skema penamaan lemah dan rentan tabrakan, sistem berisiko membuang data yang sah karena menganggapnya sebagai duplikat (*false positive*). Sebaliknya, dengan UUID murni, *idempotent consumer* dapat dengan presisi tinggi mendeteksi pengulangan pengiriman (*retry*) tanpa merusak integritas *log* yang masuk.

5. T5 (Bab 5): Kapan *Total Ordering* Tidak Diperlukan dan Pendekatan Praktisnya

Dalam sistem terdistribusi, ketiadaan jam global (*global clock*) membuat sinkronisasi urutan kejadian (*event ordering*) menjadi salah satu tantangan paling kompleks (Tanenbaum & Van Steen, 2007). *Total ordering* adalah mekanisme yang menjamin bahwa seluruh komponen di dalam sistem melihat dan memproses serangkaian pesan dengan urutan yang persis sama. Namun, *total ordering* sangat mahal secara komputasi dan memicu *overhead* komunikasi. Mekanisme ini tidak diperlukan jika *event* yang diproses bersifat independen satu sama lain, atau ketika sistem mengutamakan ketersediaan (*availability*) dan performa di mana urutan absolut tidak krusial bagi hasil akhir (Tanenbaum & Van Steen, 2007).

Pada rancangan *Pub-Sub log aggregator* ini, memaksakan *total ordering* saat data masuk ke `asyncio.Queue` akan menciptakan *bottleneck* yang menurunkan *throughput* API secara drastis. Oleh karena itu, pendekatan praktis yang diambil adalah membiarkan pesan diproses secara *out-of-order* dan mengandalkan atribut fisik, yaitu kolom timestamp berformat ISO8601 yang wajib dikirim oleh *Publisher* (terdefinisi pada model Pydantic di `src/models.py`). Sebagai lapisan tambahan, *Consumer* merekam waktu lokal server (`processed_at`) saat menyimpan data ke SQLite (`src/dedup_store.py`).

Batasan utama dari pendekatan praktis ini adalah ketidakakuratan jam fisik antar-mesin pengirim (*clock skew*). Karena jam pada mesin *source* jarang sekali tersinkronisasi secara presisi (Tanenbaum & Van Steen, 2007), timestamp bawaan *event* mungkin tidak mencerminkan urutan kausalitas secara absolut. Meskipun demikian, dalam konteks *log aggregator*, batasan ini sangat dapat diterima karena fokus utamanya adalah

membuang duplikasi secara *idempotent* daripada mengurutkan kejadian lintas-layanan secara presisi mutlak.

6. T6 (Bab 6): Identifikasi *Failure Modes* dan Strategi Mitigasi pada Aggregator

Berikut adalah draf untuk T6, yang membedah teori toleransi kegagalan (*Fault Tolerance*) dari Bab 6 buku Tanenbaum, dan menyoroti secara tajam bagaimana *database SQLite* serta volume Docker milikmu menyelamatkan sistem dari *crash*.

T6 (Bab 6): Identifikasi *Failure Modes* dan Strategi Mitigasi pada Aggregator

Dalam pengoperasian sistem terdistribusi, mengelola berbagai *failure modes* (mode kegagalan) adalah hal yang mutlak untuk menjaga keandalan. Dua jenis kegagalan yang paling umum adalah *crash failures*, di mana sebuah komponen mati atau terhenti secara tak terduga, serta *omission failures*, di mana pesan gagal terkirim atau hilang di dalam jaringan (Tanenbaum & Van Steen, 2007). Kegagalan jaringan ini lazim memicu pengirim (*Publisher*) untuk melakukan *retry* (pengiriman ulang) secara otomatis, yang justru berujung pada masuknya pesan duplikat. Selain itu, variasi latensi dapat menyebabkan pesan tiba secara *out-of-order* (tidak berurutan).

Untuk memitigasi kegagalan tersebut, proyek aggregator log ini mengimplementasikan strategi pemulihan yang berpusat pada penggunaan *durable dedup store*. Seperti yang terlihat pada `src/dedup_store.py`, aplikasi menggunakan basis data persisten SQLite dengan mode *Write-Ahead Logging* (WAL) untuk menjamin penulisan data yang aman ke *disk*. Ketika *container* Docker aggregator mengalami *crash* atau pemuatan ulang, antrean yang masih berada di dalam memori (`asyncio.Queue`) memang akan menguap. Namun, riwayat identitas log yang telah diproses tetap utuh (Tanenbaum & Van Steen, 2007).

Keandalan ini diperkuat dengan konfigurasi *volume mapping* (`./data:/app/data`) pada fail `docker-compose.yml`. Pemetaan ini memastikan pemulihan status (*state recovery*) berjalan sempurna pasca-*restart*. Saat *Publisher* melakukan *retry* untuk *event* lama yang dikira gagal, *Consumer* akan memvalidasinya ke *dedup store* lokal dan langsung membuangnya sebagai duplikat. Pendekatan ini berhasil menutupi (*masking*) kegagalan jaringan dan menjaga integritas metrik log tanpa perlu menghentikan ketersediaan layanan.

7. T7 (Bab 7): Definisi *Eventual Consistency* dan Peran *Idempotency* + *Dedup*

Eventual consistency adalah salah satu bentuk model konsistensi yang melonggarkan aturan sinkronisasi ketat; model ini menjamin bahwa jika tidak ada pembaruan baru yang masuk, maka pada akhirnya (*eventually*) seluruh komponen di dalam sistem akan mencapai status akhir yang sama dan konsisten (Tanenbaum & Van Steen, 2007).

Dalam konteks *Pub-Sub log aggregator* yang dibangun, *eventual consistency* secara natural terbentuk karena adanya arsitektur *asynchronous queue*. Ketika sebuah log diterima oleh antarmuka FastAPI melalui POST /publish (src/main.py), log tersebut ditampung terlebih dahulu di dalam *asyncio.Queue*. Pada momen ini, jika klien mengakses GET /events, log tersebut belum muncul. Terdapat jeda waktu (*latency*) hingga *background worker* (src/consumer.py) selesai menarik dan memproses pesan tersebut ke basis data. Sistem membiarkan ketidakkonsistenan sementara ini demi mempertahankan *throughput* penerimaan data yang tinggi.

Namun, model konsistensi ini sangat terancam oleh pengiriman pesan duplikat (*at-least-once delivery*). Jika *consumer* memproses setiap pesan yang ada di antrean secara buta, hasil akhirnya akan berisi data ganda, sehingga sistem gagal mencapai konsistensi yang benar. Di sinilah kombinasi *idempotency* dan *deduplication store* menjadi pilar utama. Dengan memvalidasi setiap pesan terhadap tabel persisten SQLite (src/dedup_store.py) menggunakan kombinasi (topic, event_id), *idempotent consumer* menjamin bahwa pemrosesan ulang tidak akan pernah mengubah *state* akhir penyimpanan. Berkat mekanisme ini, sebanyak apa pun duplikasi yang menumpuk di antrean, sistem dipastikan "pada akhirnya akan konsisten" secara sempurna dengan mencatat setiap log unik tepat satu kali (Tanenbaum & Van Steen, 2007).

T8 (Bab 1–7): Metrik Evaluasi Sistem dan Kaitannya dengan Keputusan Desain

Evaluasi kinerja sistem terdistribusi memerlukan pengukuran metrik yang merepresentasikan efisiensi arsitektur, komunikasi, dan toleransi kegagalannya secara holistik (Tanenbaum & Van Steen, 2007). Pada *Pub-Sub log aggregator* ini, evaluasi didasarkan pada tiga metrik operasional utama: *throughput*, latensi, dan *duplicate rate*.

Throughput mengukur jumlah *event* yang berhasil ditangani sistem dalam satu waktu. Untuk memaksimalkan metrik ini, keputusan desain yang diambil adalah menerapkan pemisahan proses (*decoupling*) menggunakan *asyncio.Queue*. Pendekatan ini berdampak langsung pada metrik latensi di sisi pengirim; ketika *Publisher* menembak POST /publish, API memberikan respons seketika tanpa menunggu operasi penulisan basis data selesai. *Trade-off* dari desain ini adalah munculnya latensi internal sementara sebelum *event* benar-benar konsisten dan dapat diakses melalui GET /events (Tanenbaum & Van Steen, 2007).

Metrik terakhir, *duplicate rate*, difasilitasi secara transparan melalui *endpoint* GET /stats yang melacak variabel *received*, *unique_processed*, dan *duplicate_dropped*. Tingginya rasio *duplicate_dropped* merepresentasikan agresifnya mekanisme *retry* akibat *omission failures* pada jaringan. Keputusan membangun *idempotent consumer* dengan *durable dedup store* (SQLite) terbukti sangat efektif dalam evaluasi ini. Sistem mampu menyerap tingkat duplikasi (*duplicate rate*) yang tinggi akibat komunikasi *at-least-once*, namun tetap menyajikan hasil akhir (*unique_processed*) yang konsisten dan akurat tanpa degradasi performa yang berarti (Tanenbaum & Van Steen, 2007).

Implementasi



Daftar Pustaka

Tanenbaum, A. S., & Van Steen, M. (2007). *Distributed systems: Principles and paradigms* (Edisi ke-2). Pearson Prentice Hall.